Business demo narrative

The point

Acme's invoice team should see faster handling without losing control of uncertainty. This demo shows three outcomes that look different on purpose: a clean paid invoice, an exception awaiting a person, and a reasoned rejection that never touches payment.

Spin up the UI

```
uv sync --python 3.12 --extra dev --extra ui
Copy-Item .env.example .env    # then set XAI_API_KEY in .env
uv run invoice-agents ui
```

The last command creates, migrates, and seeds both SQLite databases (idempotent, safe on every start) and serves the web console at http://127.0.0.1:8787. The dashboard opens with a preflight strip showing database and API-key health; run actions stay disabled until it is green. The whole demo can be driven from here: **Submit** runs any sample invoice with a live event stream, and **Reviews** holds paused cases so a named reviewer can decide and resume with one click.

Or run it in the terminal

The console and CLI share the same services, so every step below also works headless:

```
uv run python main.py --invoice_path=data/invoices/invoice_1001.txt
uv run invoice-agents process --invoice-path data/invoices/invoice_1001.txt
uv run invoice-agents batch --invoice-dir data/invoices --concurrency 2
uv run invoice-agents review list
uv run invoice-agents review decide REVIEW_ID --reviewer vp@example.com --decision REJECT --
reason "Not authorized."
uv run invoice-agents review resume CASE_ID
```

1. Prove the reasoning stack

Run `uv run invoice-agents contract --live`. The matrix establishes that AutoGen 0.7.5 and the exact xAI `grok-4.5` endpoint support typed tools, structured output, sequential iterations, specialist handoffs, persisted human pause/resume, and visible exceptions. A skipped or failed check is not a green result.

2. Let a clean invoice flow

Submit `invoice_1001.txt` from the console's **Submit** page and watch the live event stream, or run:

```
uv run python main.py --invoice_path=data/invoices/invoice_1001.txt
```

The agents extract line-level evidence, query the exact `WidgetA` and `WidgetB` rows, recompute USD 5,000 with `Decimal`, challenge the findings, approve, and write one local mock transaction. The console is brief; the full evidence and every handoff remain in the case record.

3. Make uncertainty visible

Run `INV-1002`. Its USD 15,000 amount crosses the policy threshold, `GadgetX` requests 20 units against 5 in stock, and the due date conflicts with Net 30. The Swarm stops, saves consistent state, and puts a complete package in the review queue (**Reviews** in the console, `review list` in the terminal). Nothing is paid while the reviewer is away.

Have a named reviewer reject it with a reason, then resume the case — one click in the console, or `review resume CASE_ID` in the terminal. The system finishes a successful workflow with `FinalDecision=REJECT`, retains the agents' original recommendation and the critic's findings, and proves the payment ledger has no transaction for the case.

4. Show why this is trustworthy

- `INV-1007` exposes a USD 110 declared/calculated discrepancy.
- `INV-1013` aggregates repeated lines before comparing stock and exposes a USD 50 total discrepancy.
- The TXT/PDF and JSON/PDF pairs are identity candidates, not separate payable invoices.
- A second payment attempt returns the first transaction as `DUPLICATE` instead of printing another success.
- Missing xAI, bad SQLite, corrupt input, invalid model schema, circuit-breaker exhaustion, and payment exceptions remain distinct failure states.

The prototype saves manual effort on clean work while routing judgment—not hiding it—on risky work.